Software Requirements Specification (SRS)

PROJECT TITLE: Heno Project Management Tool
PROJECT DEVELOPER: Joy Itodo
PROJECT REPOSITORY (GITHUB LINK): https://github.com/Jitodo15/Heno

Table of Contents

1. Introduction 3
1.1 Purpose 4
1.2 Document Conventions 4
1.3 Intended Audience and Reading Suggestions 4
1.4 Project Scope 4
1.5 References 5
2. Overall Description 5
2.1 Product Perspective 5
2.2 Product Features 5
2.3 User Classes and Characteristics 5
2.4 Operating Environment 5
2.5 Design and Implementation Constraints 6
2.6 User Documentation 6
2.7 Assumptions and Dependencies 6
3. System Features 6
3.1 User Authentication (High Priority) 6
3.1.1 Description and Priority 6
3.1.2 Stimulus/Response Sequences 6
3.1.3 Functional Requirements 6
3.2 Project Management (High Priority) 7
3.2.1 Description and Priority 7
3.2.2 Stimulus/Response Sequences 7
3.2.3 Functional Requirements 7
3.3 Task Management (High Priority) 7
3.3.1 Description and Priority 7
3.3.2 Stimulus/Response Sequences 7
3.3.3 Functional Requirements 7
3.4 AI Recommendation (Medium Priority) 7
3.4.1 Description and Priority 7
3.4.2 Stimulus/Response Sequences 7

3.4.3 Functional Requirements 7
3.5 AI Schedule Reorganization (Medium Priority) 8
3.5.1 Description and Priority 8
3.5.2 Stimulus/Response Sequences 8
3.5.3 Functional Requirements 8
3.6 Notifications (Low Priority) 8
3.6.1 Description and Priority 8
3.6.2 Stimulus/Response Sequences 8
3.6.3 Functional Requirements 8
3.7 Data Visualization (Medium Priority) 8
3.7.1 Description and Priority 8
3.7.2 Stimulus/Response Sequences 8
3.7.3 Functional Requirements 9
3.8 Collaboration (Medium Priority) 9
3.8.1 Description and Priority 9
3.8.2 Stimulus/Response Sequences 9
3.8.3 Functional Requirements 9
3.9 Profile Management (Low Priority) 9
3.9.1 Description and Priority 9
3.9.2 Stimulus/Response Sequences 9
3.9.3 Functional Requirements 9
3.10 Concurrency Control (High Priority) 9
3.10.1 Description and Priority 9
3.10.2 Stimulus/Response Sequences 9
3.10.3 Functional Requirements 9
4. External Interface Requirements 10
4.1 User Interfaces 10
4.2 Hardware Interfaces 10
4.3 Software Interfaces 10
4.4 Communications Interfaces 10
5. Other Nonfunctional Requirements 10

5.1 Performance Requirements 10
5.2 Safety Requirements 10
5.3 Security Requirements 11
5.4 Software Quality Attributes 11
6. Other Requirements 11
Appendix A: Glossary 11
Appendix B: Analysis Models 11
Appendix C: Issues List 13

Revision History

Name	Date	Reason for Changes	Version
Joy Itodo	Nov 14, 2025	Initial Draft of SRS	1.0

1. Introduction

This section introduces the purpose, scope, and audience of this SRS. It must be consistent with the Project Charter submitted earlier.

1.1 Purpose

This Project Charter defines the scope, objectives, deliverables, and high-level plan for the development of *HENO*. This document authorizes the project to commence and provides a foundational reference for its execution throughout the semester. It confirms our shared understanding of the project's direction and serves as an internal contract for the work to be completed.

1.2 Document Conventions

This document follows the **IEEE 830** standard for Software Requirements Specifications.

- Requirements are labeled as **FRx** (Functional Requirements) and **NFRx** (Non-Functional Requirements).
- Priority conventions: **High (H)**, **Medium (M)**, **Low (L)**.
- Typographical conventions: *Italics* for terms, **bold** for section headings, and `monospace` for code or technical keywords.

1.3 Intended Audience and Reading Suggestions

Describe the target readers of this document (developers, testers, instructors, etc.). Suggest the order in which different readers should approach the document.

- **Developers:** Should focus on Sections 2–5 for technical requirements and implementation details.
- **Testers:** Should review Section 3 (System Features) and Section 5 (Nonfunctional Requirements) for test case creation.
- **Instructors:** Should review all sections for completeness and consistency.

Note: Readers are encouraged to read sequentially, starting with the Introduction, then Overall Description, followed by System Features.

1.4 Project Scope

The ***Heno*** project is a project management tool designed to enhance collaboration and productivity through AI-powered task recommendations, schedule reorganization, and interactive data visualization dashboards. It enables users to manage projects, assign tasks, and track progress efficiently, while concurrency control ensures data integrity during collaborative

editing. The system's goals are to improve project planning accuracy, streamline teamwork, and provide clear visual insights into performance. Advanced features like offline support, accessibility enhancements, and complex third-party integrations are excluded from this version to maintain focus and ensure timely delivery.

1.5 References

- Joy Itodo. (2025). *Project Charter for Heno* [Unpublished manuscript]. School of Business and Technology, Huston-Tillotson University.
- IEEE Std 830-1998: Software Requirements Specification Standard.
- React Documentation: https://react.dev
- Node.js Documentation: https://nodejs.org

2. Overall Description

2.1 Product Perspective

Heno is a **standalone web application** that provides a comprehensive environment for managing projects and tasks. It integrates with external APIs like **Gemini LLM** (for AI-powered scheduling) and **Pexels API** (for project cover images). The frontend is built with **React.js**, and the backend uses **Node.js** with **Prisma ORM** to interact with a **PostgreSQL** database.

2.2 Product Features

- **Project Management:** Create, edit, and delete projects.
- **Task Management:** Add, assign, and update tasks dynamically.
- **AI-Powered Assistance:** Suggest tasks and reorganize schedules for better workload balance.
- **Notifications:** Real-time alerts for task updates, comments, or deadlines.
- **Data Visualization:** Interactive charts to display progress and completion rates.
- **Collaboration:** Add team members, comment on tasks, and track changes.
- **User Profiles:** Manage personal information and activity history.

2.3 User Classes and Characteristics

User Class	Description
Project Manager	Creates and oversees projects, assigns tasks, and monitors progress.
Team Member	Works on assigned tasks, updates progress and interacts via comments.
Administrator	Manages user access, permissions, and system configurations.

2.4 Operating Environment

- **Frontend:** React.js Runs on any modern browser (Chrome, Firefox, Safari, Edge).
- **Backend:** Node.js environment hosted on an online server (e.g., render).
- **Database:** PostgreSQL via Prisma ORM.
- **Communication:** HTTPS and WebSockets for secure and real-time interaction.

2.5 Design and Implementation Constraints

- Must comply with standard web security protocols.
- Backend must maintain concurrency control using lock-based mechanisms.
- System should be scalable to handle multiple concurrent users.

2.6 User Documentation

- A User Manual will be provided, covering navigation, feature usage, and troubleshooting.
- Online tooltips and help modals will guide users through complex features.

2.7 Assumptions and Dependencies

- Users have stable internet connectivity.
- External APIs (Gemini LLM, Pexels) remain functional.
- Email or notification service is available for alert delivery.

3. System Features

At least 10 system features (ideally) will be defined. Each feature should include description, priority, stimulus/response sequences, and functional requirements.

3.1 User Authentication (High Priority)

3.1.1 Description and Priority

Allows users to securely register, log in, and manage sessions. **Priority: High**

3.1.2 Stimulus/Response Sequences

User enters credentials → system verifies → grants access.

3.1.3 Functional Requirements

- **FR1:** The system shall allow users to sign up using email and password.
- **FR2:** The system shall validate credentials during login.

- **FR3:** The system shall allow password reset via email.

3.2 Project Management (High Priority)

3.2.1 Description and Priority

Users can create, edit, and delete projects. **Priority: High**

3.2.2 Stimulus/Response Sequences

User clicks “Create Project” → fills form → project added to dashboard.

3.2.3 Functional Requirements

- **FR4:** The system shall allow users to create projects with title, description, due date, and priority.
- **FR5:** The system shall store and display project details dynamically.

3.3 Task Management (High Priority)

3.3.1 Description and Priority

Allows users to create, edit, delete, assign, and track a task. **Priority: High**

3.3.2 Stimulus/Response Sequences

User clicks “Create Task” → fills form → project added to dashboard.

3.3.3 Functional Requirements

- **FR6:** The system shall allow users to create, edit, and assign tasks.
- **FR7:** Tasks shall be grouped by project and status (Upcoming, Overdue, Completed).

3.4 AI Recommendation (Medium Priority)

3.4.1 Description and Priority

Recommends tasks for users depending on the project's description. **Priority: Medium**

3.4.2 Stimulus/Response Sequences

User creates project → opens the project → clicks on “AI Recommended Tasks”.

3.4.3 Functional Requirements

- **FR8:** The system shall recommend tasks and due dates based on project goals and team workload.

3.5 AI Schedule Reorganization (Medium Priority)

3.5.1 Description and Priority

Sometimes when the user is tired and mistakenly puts the wrong date, or assigns tasks to the wrong person, the user can use this feature to reorganize it. **Priority: Medium**

3.5.2 Stimulus/Response Sequences

User opens a project → clicks on "AI help me reorganize" → accepts or rejects the suggested reorganization

3.5.3 Functional Requirements

- **FR9:** The system shall adjust task start and end dates to resolve scheduling conflicts.

3.6 Notifications (Low Priority)

3.6.1 Description and Priority

Allows users to receive comments in real time, when a task or project has been updated.
Priority: Low

3.6.2 Stimulus/Response Sequences

Task/ Project created (with user added) → added users get a notification letting them know their assigned task

3.6.3 Functional Requirements

- **FR10:** The system shall notify users in real time when a task or project is updated.

3.7 Data Visualization (Medium Priority)

3.7.1 Description and Priority

Allows users to have a visual representation of their project progress using charts like bar charts, histograms, etc. **Priority: Medium**

3.7.2 Stimulus/Response Sequences

User opens a project dashboard → user can see charts that visually describe their project progress.

3.7.3 Functional Requirements

- **FR11:** The system shall display charts showing progress and completion rates.

3.8 Collaboration (Medium Priority)

3.8.1 Description and Priority

Allows users to add comments on projects and tasks. **Priority: Medium**

3.8.2 Stimulus/Response Sequences

User opens project → clicks on a task → leaves a comment.

3.8.3 Functional Requirements

- **FR12:** Users can comment on tasks and mention teammates.

3.9 Profile Management (Low Priority)

3.9.1 Description and Priority

Allows users to update their profile information. **Priority: Low**

3.9.2 Stimulus/Response Sequences

Click on the profile icon → click on the "edit" button → lets you edit your information.

3.9.3 Functional Requirements

- **FR13:** Users can view and update their profile details.

3.10 Concurrency Control (High Priority)

3.10.1 Description and Priority

System uses a lock-based concurrency control to avoid simultaneous overwrite. **Priority: High**

3.10.2 Stimulus/Response Sequences

Two users trying to edit a task/project simultaneously → locks it for the second person → send a notification "this task/project is currently being edited, wait for some seconds" → unlocks when the first user is done -> grants access to the second user

3.10.3 Functional Requirements

- **FR14:** The system shall use lock-based concurrency control to prevent simultaneous overwrites of the same task.

4. External Interface Requirements

4.1 User Interfaces

- Modern React-based interface with sidebar navigation.
- Dashboard showing tasks under three tabs: Upcoming, Overdue, Completed.
- Modals for task editing and project details.

4.2 Hardware Interfaces

No special hardware is required. Runs on devices with a standard web browser.

4.3 Software Interfaces

- **Database:** PostgreSQL.
- **AI Service:** Google LLM.
- **Notification Service:** WebSocket or Firebase.
- **Image API:** Pexels API.

4.4 Communications Interfaces

- HTTPS for secure requests.
- WebSockets for real-time data updates.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- Average response time: ≤ 2 seconds under standard load.
- The system should support up to 10,000 concurrent users.

5.2 Safety Requirements

- Regular database backups must be scheduled.
- Critical data operations must include rollback support.

5.3 Security Requirements

- All user data encrypted in transit and at rest.
- User authentication handled with hashed and salted passwords.

5.4 Software Quality Attributes

- **Usability:** Clean, intuitive UI with clear navigation.
- **Reliability:** System uptime ≥ 99%.
- **Maintainability:** Modular codebase with version control.
- **Scalability:** Easily deployable on cloud platforms.

6. Other Requirements

- Compliance with data privacy regulations (e.g., GDPR).
- Backup and restore functionality for database.
- API keys for third-party services stored securely.

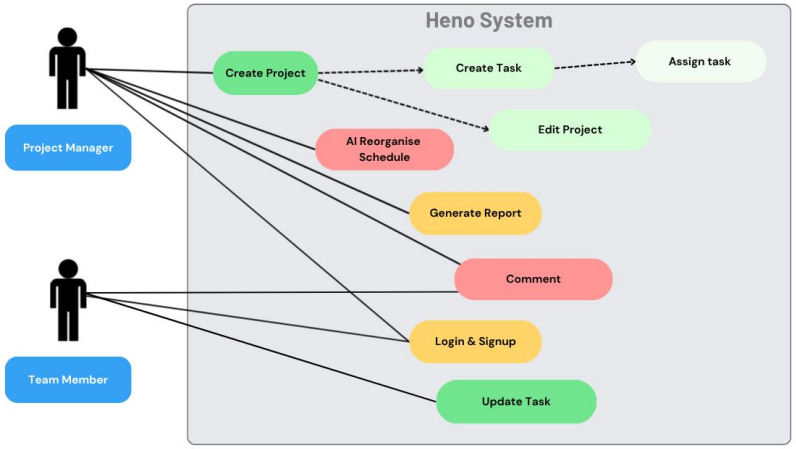
Appendix A: Glossary

Term	Definition
AI	Artificial Intelligence
CRUD	Create, Read, Update, Delete
MVP	Minimum Viable Product
UI/UX	User Interface / User Experience
LLM	Large Language Model

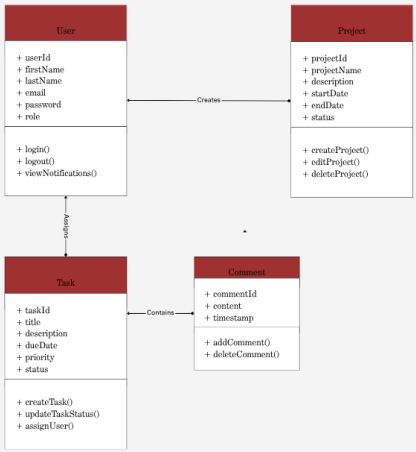
Appendix B: Analysis Models

- Object-Oriented Class Diagram
- Use Case Diagram
- Entity-Relationship Diagram

Heno UML Use Case Diagram



Heno Object-oriented Class Diagram



Appendix C: Issues List

Document any unresolved issues, TBD items, or pending decisions related to the requirements.

Issue	Description / Status
AI API dependency	Pending confirmation of model integration limits.
Real-time sync	WebSocket configuration testing.
Data visualization refinement	Chart rendering performance optimization required.